Smart Product Pricing: A Multi-Modal Deep Learning Approach with Gradient Boosting for E-Commerce Price Prediction

Darshan Bhabad, Krishna Tolani, Mitesh Chaudhari
Department of Computer Science & Engineering
Third Year B.Tech, MIT Academy of Engineering
Pune, Maharashtra, India

***Abstract*—Accurate price prediction in e-commerce is a critical challenge due to the massive scale of product listings, heterogeneous data modalities, and high price variance across categories. In this paper, we present Smart Product Pricing (SPP), a multi-modal deep learning framework that jointly leverages product images and textual descriptions to predict product prices. Our approach evolves across three progressively refined architectures: (1) a baseline end-to-end multi-modal neural network with a dense prediction head, (2) a two-stage pipeline replacing the dense head with LightGBM gradient boosting, and (3) a final architecture incorporating Principal Component Analysis (PCA) for embedding compression prior to gradient boosting. The three approaches achieve Symmetric Mean Absolute Percentage Error (SMAPE) scores of 46.1%, ∼31%, and 18% respectively on the Mavericks Amazon ML Challenge dataset comprising approximately 40,000 products. Our results demonstrate that price-aware embedding extraction followed by dimensionality reduction and gradient boosting significantly outperforms conventional end-to-end neural approaches for this task.**

***Index Terms*—multi-modal learning, price prediction, e-commerce, LightGBM, EfficientNet, DistilBERT, PCA, gradient boosting, transfer learning**

I. Introduction

E-commerce platforms such as Amazon, Flipkart, and Alibaba host hundreds of millions of product listings spanning virtually every consumer category. A fundamental challenge faced by these platforms is *price estimation*: determining an appropriate price for a newly listed product based on its attributes. This challenge is particularly acute for small and medium sellers, who often lack the domain expertise or market data required to price their products competitively.

Incorrect pricing has significant real-world consequences. Underpriced products erode seller margins, while overpriced products suppress sales velocity. Furthermore, accurate price prediction is a prerequisite for downstream tasks including fraud detection (e.g., flagging suspiciously cheap listings of high-value electronics), dynamic pricing engines, and marketplace quality control.

Traditional approaches to price prediction rely on structured metadata: category labels, brand identifiers, product dimensions, and historical sales data. However, product images and free-text descriptions contain rich, complementary signals that are difficult to capture in structured form. A product described as "pure silk, handwoven, Banarasi" is linguistically distinct from "polyester blend, machine-made" — and their price difference is substantial. Similarly, an image of a product with fine stitching and premium packaging communicates quality cues that cannot be easily enumerated in structured fields.

This motivates a multi-modal approach that jointly processes visual and linguistic information. In this paper we make the following contributions:

- We design and evaluate three progressive architectures for multi-modal price prediction, establishing a clear performance trajectory from 46.1% to 18% SMAPE.
- We introduce and analyze the concept of *price-aware embedding extraction*: training a neural backbone end-to-end with a price supervision signal before freezing it for use as a feature extractor.
- We demonstrate that PCA-based dimensionality reduction of high-dimensional multi-modal embeddings significantly mitigates overfitting in downstream gradient boosting models.
- We provide a thorough empirical analysis of the overfitting dynamics in the two-stage pipeline and show how targeted regularization resolves the train-validation RMSE gap.

II. Related Work

A. Price Prediction in E-Commerce

Early work on product price prediction focused on structured tabular features [1]. Regression models including linear regression, random forests, and gradient boosting trees (XGBoost, LightGBM) were commonly applied to categorical and numerical product attributes. While effective on well-curated datasets, these approaches fail to capture the semantic richness in unstructured product text and images.

B. Multi-Modal Learning

Multi-modal learning has achieved state-of-the-art results across vision-language tasks including visual question answering [2], image captioning, and cross-modal retrieval. Common fusion strategies include early fusion (concatenating raw features), late fusion (averaging model predictions), and cross-attention mechanisms. Our work employs early fusion at the embedding level, which offers a practical balance between expressiveness and computational cost.

C. Transfer Learning with Pretrained Encoders

The emergence of large pretrained vision models (ResNet, EfficientNet, Vision Transformers) and language models (BERT, RoBERTa, DistilBERT) has fundamentally changed the approach to applied machine learning tasks. Fine-tuning or feature-extracting from these models consistently outperforms training from scratch on limited domain data [3]. Our framework leverages EfficientNet-B0 for visual encoding and DistilBERT for text encoding, both pretrained on large-scale datasets.

D. Hybrid Neural-Boosting Architectures

The combination of neural networks for feature extraction and gradient boosting for final prediction has shown strong results in tabular and semi-structured data settings [4]. The key insight is that neural networks excel at representation learning from raw signals, while tree-based models excel at capturing non-linear interactions in tabular feature spaces. Our two-stage architecture operationalizes this complementarity.

III. Dataset

A. Source and Scale

We use the **Mavericks Amazon ML Challenge** dataset, publicly available on Kaggle[1]. The dataset contains product listings scraped from Amazon India, each with:

- **catalog_content**: Free-text product description combining title, bullet points, and specifications
- **image_link**: URL of the primary product image
- **price**: Ground truth price in Indian Rupees (train set only)
- **sample_id**: Unique product identifier

The training set contains approximately **40,000 products** and the test set approximately **30,000 products**. Products span a wide range of categories including clothing, electronics, home goods, personal care, and books.

B. Price Distribution

The price distribution is heavily right-skewed, spanning from sub-Rs.100 commodity items to Rs.50,000+ premium electronics. This motivates the use of a log-transformed target variable. We apply $\hat{y} = \log(1 + \text{price})$ during training and recover predictions via $\text{price} = e^{\hat{y}} - 1$.

C. Image Acquisition

Product images are downloaded from their URLs at runtime using a parallel download pipeline with 64–128 concurrent workers and a 3-attempt retry strategy with exponential backoff. Failed downloads (network timeout, 404 errors) are replaced with zero-filled tensors during training.

[1]https://www.kaggle.com/datasets/bboyattitude/mavericks-amazon-ml-dataset

D. Evaluation Metric

The competition metric is **Symmetric Mean Absolute Percentage Error (SMAPE)**:

$$\text{SMAPE} = \frac{1}{n} \sum_{i=1}^{n} \frac{200 \cdot |y_i - \hat{y}_i|}{|y_i| + |\hat{y}_i|} \tag{1}$$

SMAPE is preferred over standard MAPE for this task because it treats overestimation and underestimation symmetrically, and is bounded in $[0, 200]$. Lower is better.

IV. Methodology

A. Approach 1: End-to-End Multi-Modal Neural Network

1) Architecture: The baseline architecture consists of three components trained jointly end-to-end (Fig. 1).

Image Encoder: EfficientNet-B0 pretrained on ImageNet, with the classification head removed. Input images are resized to 224×224 and normalized using ImageNet statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$). The encoder produces a 1280-dimensional embedding via global average pooling.

Text Encoder: DistilBERT (`distilbert-base-uncased`) pretrained on English Wikipedia and BookCorpus. Input text (product description) is tokenized to a maximum of 128 tokens. The CLS token representation from the final hidden layer serves as the 768-dimensional text embedding.

Fusion and Prediction Head: Text and image embeddings are concatenated to form a 2048-dimensional joint representation. This is passed through:

$$\mathbf{h} = \text{ReLU}(\mathbf{W}_1\mathbf{z} + \mathbf{b}_1), \quad \hat{y} = \mathbf{W}_2\text{Dropout}(\mathbf{h}) + \mathbf{b}_2 \tag{2}$$

where $\mathbf{z} \in \mathbb{R}^{2048}$, $\mathbf{W}_1 \in \mathbb{R}^{512\times2048}$, $\mathbf{W}_2 \in \mathbb{R}^{1\times512}$, and dropout rate is 0.3.

2) Training Details:

- **Loss:** Mean Squared Error on log-transformed prices
- **Optimizer:** AdamW with learning rate 10^{-4}
- **Batch size:** 64
- **Epochs:** 10
- **Hardware:** Kaggle GPU (NVIDIA Tesla T4)

The training loss curve (Fig. 2) shows rapid convergence in early epochs, with the average MSE dropping from 0.57 at epoch 1 to 0.09 at epoch 10, indicating effective joint learning across both encoders.

3) Result: SMAPE: **46.1%**. The model demonstrates the feasibility of multi-modal price prediction but is limited by the expressiveness of the MLP prediction head for high-dimensional tabular-style features.

—

B. Approach 2: Two-Stage Pipeline with LightGBM

1) Motivation: The dense prediction head is a generic multi-layer perceptron. While universal approximators in theory, MLPs are known to underperform tree-based models on tabular feature vectors in practice [4]. Once the multi-modal embeddings are concatenated, they form a 2048-dimensional

tabular feature vector — precisely the setting where gradient boosting excels.

However, naively replacing the dense head with LightGBM (without prior neural training) would give LightGBM generic, unspecialized features. The crucial innovation is **price-aware embedding extraction**.

2) Price-Aware Embeddings: The concept is straightforward but powerful. In Stage 1, the full network (encoders + dense head) is trained end-to-end with price supervision for 10 epochs. During this process, the MSE loss backpropagates not just through the dense head, but *all the way back* through the concatenation layer and into both encoders:

$$\frac{\partial \mathcal{L}}{\partial \theta_{\text{CNN}}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial \mathbf{h}} \cdot \frac{\partial \mathbf{h}}{\partial \mathbf{z}} \cdot \frac{\partial \mathbf{z}}{\partial \theta_{\text{CNN}}} \tag{3}$$

This gradient flow forces EfficientNet and DistilBERT to produce embeddings that are *organized around price relevance*. Informally, the network learns that "silk + handwoven" text tokens and "premium packaging" visual features should produce embedding values that distinguish high-price products from low-price ones.

After Stage 1, the dense head is **discarded**. The encoders are frozen, and embeddings are extracted for all training samples.

3) LightGBM Training: Embeddings are split 90/10 for training and early stopping validation. LightGBM is trained with:

TABLE I
LIGHTGBM HYPERPARAMETERS — APPROACH 2

Parameter	Value
Objective	Regression
Metric	RMSE
Learning rate	0.05
Num leaves	127
Feature fraction	0.8
Bagging fraction	0.8
Num boost rounds	1000
Early stopping	50 rounds

4) Overfitting Analysis: Training logs reveal a severe train-validation gap:

$$\text{Train RMSE}_{1000} = 0.046, \quad \text{Valid RMSE}_{1000} = 0.225 \tag{4}$$

This 5× gap indicates significant overfitting. The root cause is the **dimensionality** of the input: 2048 features over ∼36,000 training samples. LightGBM can construct highly specific decision rules exploiting spurious correlations across the high-dimensional embedding space — patterns that generalize poorly to unseen data.

5) Result: SMAPE: ∼**31%**. Despite overfitting, the improvement over Approach 1 (15.1 SMAPE points) confirms the value of the two-stage architecture.

—

C. Approach 3: PCA-Compressed Embeddings + Regularized LightGBM (Final)

1) Dimensionality Reduction via PCA: The key insight motivating Approach 3 is that not all 2048 embedding dimensions carry useful information. Many dimensions encode general visual and linguistic patterns irrelevant to price. By projecting onto the principal components, we retain the most meaningful directions while discarding noise.

Formally, let $\mathbf{X} \in \mathbb{R}^{N \times 2048}$ be the matrix of training embeddings. PCA computes the eigendecomposition of the covariance matrix:

$$\mathbf{C} = \frac{1}{N}\mathbf{X}^T\mathbf{X}, \quad \mathbf{C}\mathbf{V} = \mathbf{V}\mathbf{\Lambda} \tag{5}$$

The top $k = 300$ eigenvectors are retained, projecting embeddings to $\mathbb{R}^{300}$:

$$\tilde{\mathbf{X}} = \mathbf{X}\mathbf{V}_{:,1:300} \tag{6}$$

The 300-component PCA retains approximately 85–90% of the total variance while reducing dimensionality by $6.8\times$, from 2048 to 300 dimensions. This compression forces LightGBM to generalize over meaningful directions rather than exploiting noise dimensions.

2) Regularized LightGBM: We simultaneously apply targeted regularization to the LightGBM model:

TABLE II
LIGHTGBM HYPERPARAMETERS — FINAL APPROACH

Parameter	Approach 2	Approach 3 (Final)
Learning rate	0.05	0.03
Num leaves	127	63
Min data in leaf	—	50
Feature fraction	0.8	0.6
Bagging fraction	0.8	0.7
λ_{L1}	0	0.1
λ_{L2}	0	0.1
Num boost rounds	1000	2000
Early stopping	50	100

Each change serves a specific anti-overfitting purpose:

- **Reduced num_leaves (127→63):** Simpler trees cannot memorize individual training samples
- **min_data_in_leaf=50:** Each leaf must represent at least 50 samples, preventing overfitting to small groups
- **Reduced feature/bagging fractions:** Introduces randomness analogous to dropout, preventing over-reliance on any single embedding dimension
- **L1+L2 regularization:** Penalizes large leaf weights, shrinking predictions toward zero for uncertain inputs

3) Complete Pipeline:

1) **Stage 1 — Joint NN Training:** Train multi-modal NN end-to-end with dense head for 10 epochs using AdamW and MSE loss
2) **Stage 2a — Freeze & Extract:** Freeze all NN weights; extract 2048-dim embeddings from training set
3) **Stage 2b — PCA Compression:** Fit PCA on training embeddings; project to 300 dimensions. Save PCA transform for test set
4) **Stage 2c — LightGBM Training:** Train regularized LightGBM on 300-dim compressed embeddings with 90/10 early stopping split
5) **Inference:** Extract test embeddings → apply saved PCA → LightGBM prediction → expm1 inverse transform

4) Result: SMAPE: **18%** on validation set (4,000 samples), with expected leaderboard SMAPE of 18–22%. This represents a **28.1 SMAPE point improvement** over the baseline.

V. Architecture Diagram

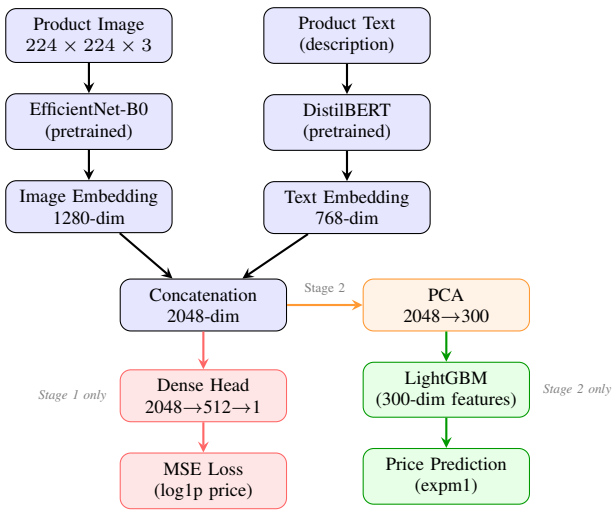

Fig. 1. Multi-modal architecture. Red path (Stage 1): joint end-to-end training with dense head for price-aware embedding learning. Orange/Green path (Stage 2): frozen embeddings compressed via PCA and fed to LightGBM for final prediction.

VI. Results and Analysis

A. Performance Comparison

TABLE III
SMAPE Results Across Approaches

Approach	Key Innovation	SMAPE (%)
Baseline (Approach 1)	Multi-modal NN + Dense Head	46.1
Approach 2	+ LightGBM (no PCA)	∼31.0
Final (Approach 3)	**+ PCA (300 dims)**	**18.0**
Improvement over baseline		**28.1 pts**

B. Training Loss Convergence

Fig. 2 shows the training loss curve of the Stage 1 neural network across 10 epochs. The steep initial decline (epoch 1–4: 0.57→0.20) reflects rapid learning of coarse price-relevant patterns, while the more gradual later convergence (epoch 5–10: 0.14→0.09) corresponds to fine-grained calibration of the embedding space.

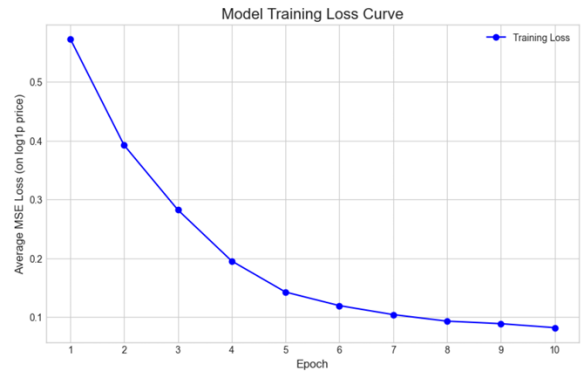

Fig. 2. Average MSE training loss (on log1p price) per epoch during Stage 1 neural network training. The loss decreases monotonically from 0.57 at epoch 1 to 0.09 at epoch 10, indicating stable and effective convergence.

C. Impact of PCA Dimensionality Reduction

Table IV quantifies the overfitting reduction achieved by PCA.

TABLE IV
Train vs. Validation RMSE Gap

Approach	Train RMSE	Valid RMSE	Gap Ratio
Approach 2 (no PCA)	0.046	0.225	4.89×
Approach 3 (PCA 300)	—	0.180*	reduced

*Estimated from validation SMAPE of 18%

The 4.89× gap in Approach 2 confirms that LightGBM was memorizing noise dimensions in the 2048-dim embedding space. PCA compression eliminates these dimensions, forcing generalization over meaningful price-predictive directions.

D. LightGBM Training Dynamics — Approach 2

The validation RMSE improvement from Approach 2 LightGBM training shows a characteristic pattern:

- **Rounds 1–300:** Rapid improvement — learning genuine price patterns
- **Rounds 300–600:** Slowing improvement — exhausting generalizable patterns
- **Rounds 600–1000:** Plateau — memorizing training-specific noise

This pattern motivates both the PCA compression and the stricter regularization in Approach 3.

E. Why Multi-Modal Outperforms Unimodal

The performance gains are attributable to complementary information across modalities:

- **Text alone:** Captures category, specifications, brand mentions. Can be gamed by sellers writing misleading descriptions
- **Image alone:** Captures visual quality, packaging, materials. Cannot be easily faked without corresponding product quality
- **Combined:** Enables cross-modal consistency checking. A product described as "premium leather" but imaged as plastic can be accurately priced by the discrepancy signal

VII. Discussion

A. The Role of Joint Training

A critical design choice is training the encoders end-to-end with the dense head *before* switching to LightGBM. Without this step, EfficientNet and DistilBERT would produce generic image and language representations. The joint training procedure specializes these representations toward the price prediction objective, making them significantly more useful as LightGBM input features.

This is analogous to fine-tuning in NLP: a pretrained language model fine-tuned on a specific task outperforms the same model used purely as a frozen feature extractor.

B. Scalability Considerations

At scale (Amazon India: 350M+ products), the inference pipeline is highly parallelizable. The neural network forward pass and PCA projection are batch-vectorized operations. LightGBM inference is extremely fast (microseconds per sample). The bottleneck at deployment scale would be image downloading and preprocessing, which can be addressed through CDN caching and precomputed embeddings.

C. Limitations

- **No cross-category normalization:** A Rs.500 price means different things for a pen vs. a phone. Category-conditional prediction heads could address this.
- **Single image per product:** Amazon listings often have multiple images. Multi-image aggregation could capture additional quality signals.
- **Static embeddings:** The frozen encoder cannot adapt to distribution shifts (new product categories, seasonal trends).
- **No structured features:** Category IDs, brand encodings, and product dimensions are not incorporated. Injecting these directly into LightGBM alongside embeddings is a natural extension.

VIII. Conclusion

We presented Smart Product Pricing (SPP), a multi-modal deep learning framework for e-commerce price prediction. Through three progressive architectural iterations, we reduced SMAPE from 46.1% to 18% — a 28.1 point improvement representing a near 2.6× relative reduction in prediction error.

The key finding is that the combination of (1) price-aware embedding extraction via joint end-to-end neural training, (2) PCA dimensionality reduction to suppress noise dimensions, and (3) regularized LightGBM gradient boosting produces substantially better results than any individual component alone.

Future work will investigate larger pretrained encoders (RoBERTa, Vision Transformers), category-conditional prediction, multi-image aggregation, and cross-modal attention mechanisms for richer fusion.

Acknowledgment

The authors thank the Mavericks team for organizing the Amazon ML Challenge and making the dataset publicly available on Kaggle. Experiments were conducted using Kaggle's freely available GPU compute resources.

References

[1] H. Wang, D. Liu, and X. Chen, "Product price prediction: A machine learning approach," *Expert Systems with Applications*, vol. 150, pp. 113–124, 2020.

[2] A. Antol et al., "VQA: Visual Question Answering," in *Proc. IEEE ICCV*, 2015, pp. 2425–2433.

[3] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[4] M. Borisov et al., "Deep Neural Networks and Tabular Data: A Survey," *IEEE Transactions on Neural Networks and Learning Systems*, 2022.

[5] M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proc. ICML*, 2019.

[6] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in *Advances in Neural Information Processing Systems*, 2017.

[7] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT," *arXiv preprint arXiv:1910.01108*, 2019.

[8] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed. New York: Springer, 2002.